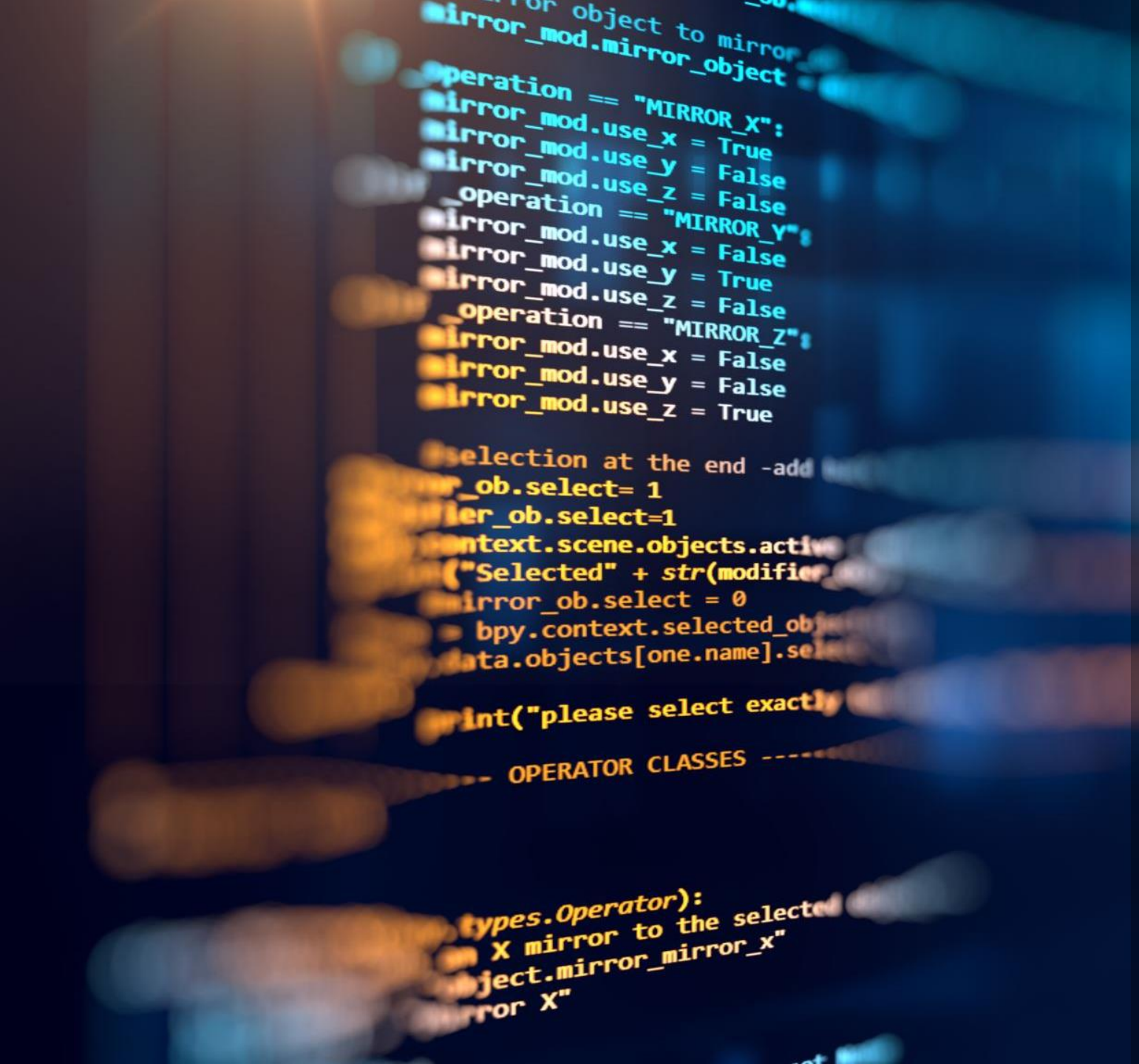

UT1. Introducción al desarrollo del software.

Git y GitHub

¿Qué es el Software y un Programa?

1. **Software:** El **equipamiento lógico** o conjunto de programas, procedimientos y documentación que permiten al hardware realizar tareas.
2. **Programa:** Un conjunto de instrucciones escritas en un lenguaje de programación, que, una vez ejecutadas, realizan tareas específicas.
3. **Aplicación Informática:** Conjunto de programas con una finalidad más o menos concreta (ej. paquete de ofimática)

Clasificación del Software (Por Función)

1. **Software de Sistemas (Base):**

La Traducción: Hardware vs. Software

1. **Arquitectura Von Neumann (1946):** Primera con programa almacenado, clave para la relación.
2. **El Lenguaje del Hardware:** Solo entiende **código binario** (secuencias de 0 y 1).
3. **La Traducción:** Proceso de transformar las sentencias de alto nivel (fácil para humanos) a código binario (comprensible por el hardware).
4. **Rol del SO:** El Sistema Operativo actúa como intermediario invisible, gestionando los recursos de la CPU, Memoria y E/S para las aplicaciones.

Licencias de Software y Propiedad Intelectual

1. **Licencia:** Contrato legal entre desarrollador y usuario sobre el uso del software.
2. **Copyright (Derecho de Autor):** Protección legal de la autoría de obras originales.
3. **Patente:** Garantiza derechos exclusivos sobre un nuevo producto o proceso (por tiempo limitado).
4. **Software Propietario (Privativo):** Uso, redistribución o modificación **prohibidos** o requieren autorización.

Software Libre y sus Implicaciones

Software Libre: Garantiza 4 libertades esenciales:

1. Usarlo.
2. Estudiar cómo funciona (acceso al código fuente).
3. Redistribuir copias.
4. Modificarlo y redistribuir la versión mejorada.

Concepto Clave: Copyleft: Principio que busca garantizar que las versiones modificadas o redistribuidas **sigan siendo libres** (no se conviertan en privativas).

Otros Tipos de Software por Licencia

1. **Freeware:** Software gratuito, pero el código fuente no está disponible y no se permite la modificación.
2. **Shareware:** Permite redistribuir copias, pero requiere pago por Licencia de uso continuado después de un periodo de prueba.
3. **Trial (Demostración):** Versión de un programa pago, con todos los recursos activos, pero por un tiempo determinado.
4. **Adware:** Software que descarga publicidad; los anuncios desaparecen al pagar por la versión comercial.
5. **Dominio Público:** Sin licencia o autor desconocido; no protegido por Copyright.

Introducción al Ciclo de Vida del Software (CVS)

Definición: El conjunto de fases por las que pasa un sistema desde su concepción hasta su retiro o reemplazo.

Fases (Los 6 Pasos):

1. Análisis (**Qué**).
2. Diseño (**Cómo**).
3. Codificación (Hacerlo).
4. Pruebas (Comprobar).
5. Explotación (Usarlo).
6. Mantenimiento (Evolucionar).

Tarea Transversal: La **Documentación** debe realizarse en cada etapa.

Modelos Rígidos del CVS: El Enfoque Cascada

1. **Cascada Clásico:** * Secuencial, requiere finalizar una etapa para empezar la siguiente.
    * **Limitación:** Presupone que no habrá errores ni cambios. Inaplicable.
2. **Cascada con Realimentación:** * Permite volver a la etapa anterior para corregir o modificar.
    * **Ideal para:** Proyectos rígidos, con requisitos estables y claros.

Modelos Evolutivos del CVS: Adaptación al Cambio

Objetivo: Tienen en cuenta la naturaleza cambiante del software y permiten entregar versiones cada vez más completas.

1. **Iterativo Incremental:** * No necesita conocer todos los requisitos al inicio. * Permite la entrega temprana al cliente de partes operativas.

    * **Riesgo:** Difícil estimación del esfuerzo total (puede que no se acabe nunca).

2. **Modelo en Espiral:** * Combina el modelo Incremental con el **análisis de riesgos** en cada ciclo (espiral).

    * **Resultado:** Versiones que incrementan la funcionalidad en cada espiral.

Fase 1: Análisis (Deducción de Requisitos)

Objetivo: Comprender el problema a resolver.

Foco: El **¿Qué?** debe hacer el sistema.

Salida Principal: Especificación de Requisitos Software (ERS).

Tipos de Requisitos:

1. **Funcionales (RF):** Describen los servicios que el sistema debe proporcionar (ej. "el sistema debe permitir pagar con tarjeta").
2. **No Funcionales (RNF):** Restricciones que afectan al sistema (ej. rendimiento, seguridad, entorno).

Técnicas de Análisis para Modelar Requisitos

1. **Diagrama de Flujo de Datos (DFD):** * Representa el movimiento de los datos (flechas) entre procesos (burbujas), almacenes y entidades externas.
2. **Casos de Uso (UML):** * Describe cómo se usa el software en una situación particular (escenarios). * Se centra en la interacción entre el usuario (actor) y el sistema.
3. **Diagrama Entidad-Relación (DER):** * Modelo conceptual que define las entidades y sus relaciones (el modelo de datos).

Fase 2: Diseño (Traduciendo el ¿Qué? al ¿Cómo?)

Objetivo: Traducir los requisitos en una representación de software (la arquitectura).
Foco: El **¿Cómo?** se va a resolver el problema.

Tareas Clave:

1. **Diseño Arquitectónico:** Definir la estructura del software (módulos, conectores).
2. **Diseño de Datos:** Transformar el DER/DD en estructuras de datos reales (bases de datos, tablas).
3. **Diseño de la Interfaz:** Formatos de pantalla y experiencia de usuario.
4. **Selección de Tecnologías:** Elegir el lenguaje de programación, SGBD, etc.

Fase 3: Codificación (Construcción del Código)

Objetivo: Traducir el diseño a una forma legible por la máquina (código ejecutable).
Conceptos Clave:

1. **Código Fuente:** Lo que escribe el programador.
2. **Código Objeto:** Código intermedio (bytecode).
3. **Linker (Enlazador):** Une los archivos de código objeto y bibliotecas para crear el **Código Ejecutable**.

Calidad del Código: Debe ser legible, correcto, eficiente, modular y, sobre todo, **portable**.

Obtención de Código: Compilación vs. Interpretación

1. **Compilación:**

    * **Proceso:** Traduce **todo** el código fuente a código objeto en un solo paso.

    * **Velocidad:** Rápida ejecución (lenguajes como C++).

2. Interpretación:

    * **Proceso:** Traduce y ejecuta **línea a línea** simultáneamente (no crea código objeto intermedio).

    * **Ventaja:** Detección de errores más detallada (lenguajes como Python).

3. Entorno de Ejecución (Runtime Environment):

    * **MV + API's:** Conjunto de bibliotecas necesarias para la ejecución.

    * **Función:** Depurar (debug) el programa, comprobando errores **semánticos** en tiempo de ejecución.

Fase 4: Pruebas (Verificación y Validación)

1. **Verificación:** ¿Se está construyendo el producto **correctamente**? (Se centra en el diseño interno).

   Técnica: Caja Blanca (examinar el código fuente).

2. Validación: ¿El producto se ajusta a los **requisitos del cliente**? (Se centra en la funcionalidad).

   Técnica: Caja Negra (probar entradas y salidas sin ver el código).

3. Herramienta: Caso de Prueba: Documento que especifica la entrada, la salida esperada y las condiciones para la prueba.

Fase 5 y 6: Documentación y Explotación

Documentación:

* **Guía Técnica:** Para programadores (facilita el desarrollo y el mantenimiento).

* **Guía de Uso:** Para usuarios finales (describe la funcionalidad).

* **Guía de Instalación:** Para personal informático (puesta en marcha).

Explotación:

* Momento en que los usuarios comienzan a usar la aplicación.

* **Beta Test:** Pruebas finales realizadas en los propios equipos del cliente y bajo cargas de trabajo normales, antes del lanzamiento final.

Fase 7: Mantenimiento y Herramientas

Mantenimiento (El 60% del Esfuerzo):

* **Correctivo:** Solucionar errores.

* **Adaptativo:** Adaptarse a cambios de entorno/hardware.

* **Perfectivo:** Mejorar la funcionalidad, eficiencia o legibilidad del código.

- **Evolutivo:** Añadir nuevas necesidades del cliente.

Herramientas de Apoyo:

- **IDE (Entorno Integrado de Desarrollo):** Conjunto todo-en-uno (editor, compilador, depurador).
- **Framework:** Conjunto de bibliotecas/esquemas que dan una estructura de trabajo predefinida.
  - **Riesgo del Framework:** Gran dependencia del código (si se cambia, hay que reescribir gran parte de la app).